



Based on the materials provided, here's a comprehensive set of revision notes for the class session on recursion, focusing on the Tower of Hanoi problem and Power Function.

Recursion: Tower of Hanoi and Power Function

Tower of Hanoi

Problem Overview

The Tower of Hanoi is a classic problem in which you are given three rods and n disks of different sizes. The objective is to move all the disks from the source rod (A) to the destination rod (C) using the auxiliary rod (B), following these rules:

1. Only one disk can be moved at a time.
2. Each move consists of taking the upper disk from one of the stacks and placing it on top of another stack or on an empty rod.
3. No larger disk may be placed on top of a smaller disk.

Recursive Solution Strategy

1. **Move $(n-1)$ disks from A to B, using C as temporary storage.**
2. **Move the n th disk (largest) directly from A to C.**
3. **Move the $(n-1)$ disks from B to C, using A as temporary storage.**

This strategy effectively breaks down the problem into smaller subproblems, which is the fundamental principle of recursion

【6:15+transcript.txt】 【6:17+typed.md】 .

Recursive Function Pseudocode

```
void TOH(int n, char A, char B, char C) {  
    if (n == 0) return;
```



}

Time and Space Complexity

- **Time Complexity:** $O(2^n)$ because the function calls itself twice for $(n-1)$ disks plus some constant time operations.
- **Space Complexity:** $O(n)$, due to the height of the recursive call stack

【6:16+transcript.txt】 【6:17+typed.md】 .

Power Function Using Recursion

Problem Overview

Calculate a^n using recursive methods. This problem illustrates the application of recursion in computational mathematics to efficiently compute powers of numbers.

Recursive Solution Strategy

1. **Base Case:** If $n == 0$, return 1 (since any number raised to the power of 0 is 1).
2. **Recursive Case:**
 - If n is even, compute $\text{pow}(a, n/2)$ and then square the result.
 - If n is odd, compute $a * \text{pow}(a, n-1)$.

Recursive Function Pseudocode

```
int power(int a, int n) {
    if (n == 0) return 1;
    int temp = power(a, n / 2);
    if (n % 2 == 0) {
        return temp * temp;
    } else {
        return a * temp * temp;
    }
}
```

Time and Space Complexity



- **Space Complexity:** $O(\log n)$, reflecting the depth of the recursion stack
【6:14+typed.md】 【6:18+transcript.txt】 .

Implementation Notes

- The recursive tree structure reflects the splitting and merging process of power computation.
- The algorithm efficiently manages time complexity by reducing the problem size logarithmically at each step 【6:12+transcript.txt】
【6:18+transcript.txt】 .

These notes provide a detailed understanding of the recursive approach to solving classic problems, illustrating key principles of recursion in computational problems.